Pitch-Level Simulation via TransFusion for Empirical Win-Probability Estimation in Baseball

Justin Fung
Carnegie Mellon University
justinfu@andrew.cmu.edu

Saagar Shah
Carnegie Mellon University
saagarsh@andrew.cmu.edu

1 Introduction

Prediction markets for sports betting require accurate, well-calibrated probability estimates for outcomes such as game winners and total runs scored. We study whether these estimates improve when we model a game's intra-game dependencies and simulate its remaining trajectory from the current state, rather than predicting the outcome directly from a summary of that state. The central intuition is that a state-summary classifier compresses the remaining uncertainty of a game into a single point estimate, whereas trajectory simulation preserves a distribution over plausible continuations.

We focus on baseball because its discrete, pitch-by-pitch structure makes it especially well suited for sequential generative modeling. Each pitch produces a categorical outcome together with continuous measurements such as velocity, spin rate, and launch angle, allowing us to treat a game as a richly annotated sequence and test whether pitch-level generation yields better probability estimates than a direct state-summary predictor.

Our dataset consists of historical pitch-level MLB Statcast data augmented with broader game context, including team records, playoff probabilities, weather conditions, and other situational attributes. [MLBnd] Our model adapts TransFusion [Zho+24], a hybrid Transformer-diffusion architecture, to autoregressively generate future pitch sequences together with the continuous features of each pitch. As a baseline, we compare against a direct win-probability classifier that predicts the game outcome from a summary of the current state. We evaluate both approaches using accuracy, Brier score, and log loss.

Our central claim is that generating the remaining pitch sequence at this level of granularity yields more robust and better-calibrated probability estimates than a state-summary approach, because it captures temporal and in-game dependencies that a single-point predictor cannot represent. Empirically, TransFusion outperforms the baseline MLP on accuracy and Brier score across most context innings, while exhibiting a late-game calibration failure in log loss.

2 Related Work

Brill et al. showed that win-probability estimators trained on observational play-by-play data are intrinsically high-variance because trajectories within a game share outcomes, inflating bias and reducing effective sample size. Our results reinforce their finding of late-game overconfidence in estimation. [BYW25]

Cui et al. proposed CaDiRec, which uses a bidirectional Transformer and diffusion model to generate context-conditional augmentations for contrastive learning on sequential data, outperforming both random-augmentation and prior diffusion-based baselines. This motivates our use of context-aware embeddings in pitch-level encoders for Monte Carlo simulation. [Cui+25]

Zhou et al. introduced TransFusion, a hybrid Transformer-Diffusion architecture that jointly models continuous and categorical variables. Sikder et al. extended this to long, high-fidelity time-series

10707 Class Project (2026).

generation with strong improvements over standard baselines. We adapt this architecture to simulate realistic pitch-by-pitch baseball trajectories for our Monte Carlo forecasts. [Zho+24; SRH23]

Stevens et al. proposed sequential posterior sampling for diffusion models, leveraging transition dynamics to initialize reverse trajectories at lower noise scales and drastically reduce sampling cost. This is directly relevant to our setting, where completing a game requires autoregressively generating hundreds of pitches per simulation. [Ste+25]

3 Methodology

3.1 TransFusion Implementation [Zho+24]

Our implementation adapts the Transfusion framework—which was originally proposed for multi-modal image-text generation—to the structured sequential domain of baseball pitch modeling. The original Transfusion architecture applies a discrete language modeling loss to text tokens and a continuous diffusion loss to image patches within a single unified Transformer, demonstrating that both modalities can be learned jointly without interference. We preserve this core design principle while replacing the image-text modality pair with pitch physics (continuous) and pitch outcomes and types (discrete), and replacing the bidirectional Transformer with a causal encoder appropriate for autoregressive game simulation.

3.1.1 Architecture Overview

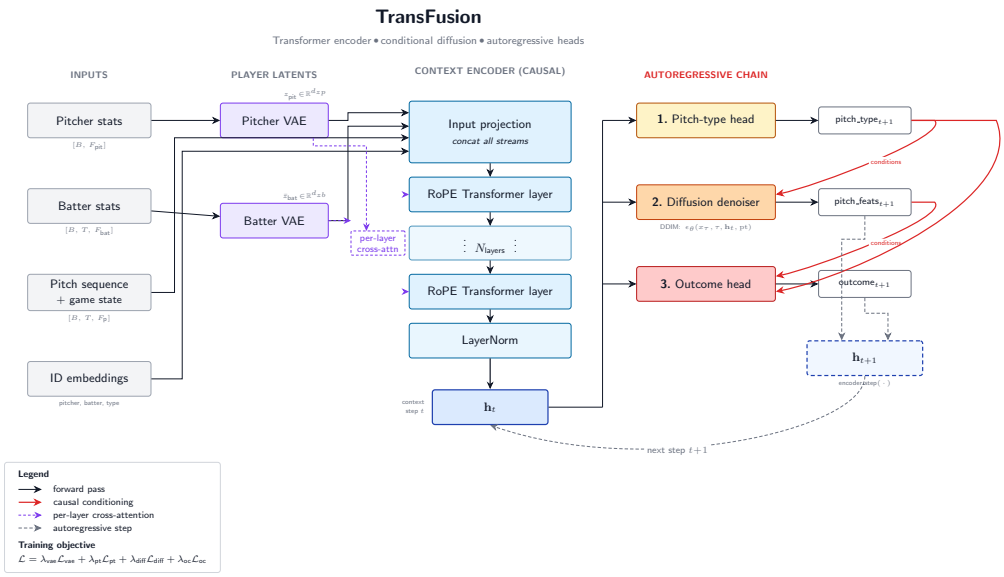

Figure 1: TransFusion architecture. Player VAEs produce latent representations that are fed into a causal Transformer encoder along with the pitch sequence and ID embeddings. The context vector $\mathbf{h}_t$ conditions an autoregressive chain of three heads: pitch-type, diffusion denoiser (DDIM), and outcome.

TransFusion consists of three jointly trained components: a causal Transformer context encoder, a diffusion-based continuous feature decoder, and dual discrete classification heads. The context encoder produces a causal representation $\mathbf{h}_t \in \mathbb{R}^d$ at each pitch position t, encoding the full history of observed pitches up to and including position t. The diffusion decoder uses $\mathbf{h}_{t-1}$ as conditioning to generate the 20-dimensional continuous feature vector of pitch $t+1$. The classification heads use $\mathbf{h}_t$ to predict the categorical pitch type and pitch outcome of pitch $t+1$. All three components share the same Transformer backbone and are trained end-to-end with a single weighted loss.

Our final model consists of the following hyperparameters: $d = 256$, $L = 6$ layers, and $H = 8$ attention heads (10.1M parameters). We incorporated a temperature parameter of $T = 0.85$ to

promote more realistic simulations. We explored increasing width to $d = 512$ (TransFusion-Wide, 22.7M parameters), increasing depth to $L = 8$ layers (TransFusion-Deep, 12.0M parameters), and a large configuration with $d = 512$, $L = 16$, $H = 16$ (TransFusion-Large, 54.2M parameters) but observed little to no improvement in validation loss along with increased overfitting.

3.1.2 Causal Masking and the Leakage Fix

The original Transfusion architecture uses a mixed attention mask: text tokens attend causally to all preceding tokens, while image patches attend fully to one another within each image. In our setting, strict causal masking is applied uniformly across the entire sequence, since pitch t's outcome must not influence the representation used to predict pitch t's own attributes.

This requirement motivated a critical implementation fix relative to a naive application of the Transfusion framework. An initial implementation predicted the outcome of pitch t from context vector $\mathbf{h}_t$, which directly embedded pitch t's own outcome as part of its per-step input. This created trivial self-prediction leakage: the model could achieve low outcome cross-entropy by simply reading the outcome embedding it had already consumed, without learning any predictive structure. The fix shifts both classification heads by one position:

$$\mathcal{L}_{\text{pt}} = \text{CE}\big(f_{\text{pt}}(\mathbf{h}_t),\ \text{type}_{t+1}\big) \tag{1}$$

$$\mathcal{L}_{\text{oc}} = \text{CE}\big(f_{\text{oc}}(\mathbf{h}_t),\ \text{outcome}_{t+1}\big) \tag{2}$$

so that context $\mathbf{h}_t$ — which encodes pitches $1, \ldots, t$ — is used to predict the attributes of pitch $t + 1$, which are not yet embedded in $\mathbf{h}_t$. This is consistent with the diffusion head, which also uses $\mathbf{h}_t$ to generate the continuous features of pitch $t + 1$, and eliminates the self-prediction shortcut entirely.

3.1.3 Diffusion Decoder

The diffusion decoder adapts the standard DDPM framework [HJA20] to conditional generation of pitch physics features. We use a cosine noise schedule [ND21] with $T = 50$ diffusion steps, which we found to provide smoother noise interpolation than the linear schedule used in the original DDPM, particularly important for the bounded physical feature distributions (e.g., release speed in the range 70–100 mph).

At training time, for each pitch $t + 1$ in the sequence, a random timestep $\tau \sim \text{Uniform}(0, T)$ is sampled and the continuous features of the target $\mathbf{x}_0 \in \mathbb{R}^{20}$ are noised:

$$\mathbf{x}_\tau = \sqrt{\bar{\alpha}_\tau}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_\tau}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{3}$$

The denoiser ϵ_θ is a two-layer MLP with SiLU activations and LayerNorm that receives the concatenation of three projected inputs: the noised pitch features $\mathbf{x}_\tau$, the diffusion timestep embedding τ, and the encoder context $\mathbf{h}_t$, each projected to $\mathbb{R}^d$, together with a pitch type embedding that conditions the denoiser on the discrete pitch category being generated.

At inference, we use DDIM sampling [SME21] with 10 denoising steps rather than the full 50-step reverse process. DDIM's deterministic sampling trajectory allows a $5\times$ reduction in denoising steps with minimal degradation in sample quality, which is critical for computational tractability during Monte Carlo simulation where the denoiser is called once per pitch per simulation trajectory.

3.2 Training

3.2.1 Training Objective and Loss Weights

The joint training objective is:

$$\mathcal{L} = \lambda_{\text{diff}}\,\mathcal{L}_{\text{diff}} + \lambda_{\text{pt}}\,\mathcal{L}_{\text{pt}} + \lambda_{\text{oc}}\,\mathcal{L}_{\text{oc}} + \lambda_{\text{vae}}\,\mathcal{L}_{\text{vae}} \tag{4}$$

where $\mathcal{L}_{\text{diff}}$ is the noise-prediction MSE, $\mathcal{L}_{\text{vae}}$ is the VAE loss, and $\mathcal{L}_{\text{pt}}$, $\mathcal{L}_{\text{oc}}$ are cross-entropy losses for pitch type and outcome classification with padding positions excluded via `ignore_index`. We set $\lambda_{\text{diff}} = 0.2$, $\lambda_{\text{vae}} = 0.1$, $\lambda_{\text{pt}} = 0.4$, $\lambda_{\text{oc}} = 0.4$, upweighting the classification heads relative to the vae and diffusion decoder. This reflects the observation that the diffusion loss converges quickly (reaching 0.18–0.20 within the first few epochs) while the classification losses, particularly the outcome head, are the binding constraint on simulation quality. By allocating more gradient signal

to the classification heads, we encourage the Transformer context encoder to learn features that are more predictive of pitch outcomes and types, which directly improves the realism of generated game trajectories.

We train with AdamW using transformer-optimized hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$, weight decay 10^{-4}) following the original Attention is All You Need recommendation [Vas+17] for transformer training stability. A linear warmup over 1000 steps is followed by cosine decay to 1% of the peak learning rate, and gradients are clipped at norm 1.0. We also incorporate curriculum learning, first training the VAE, then training the Pitch type head and diffusion decoder along with the VAE, then training all components.

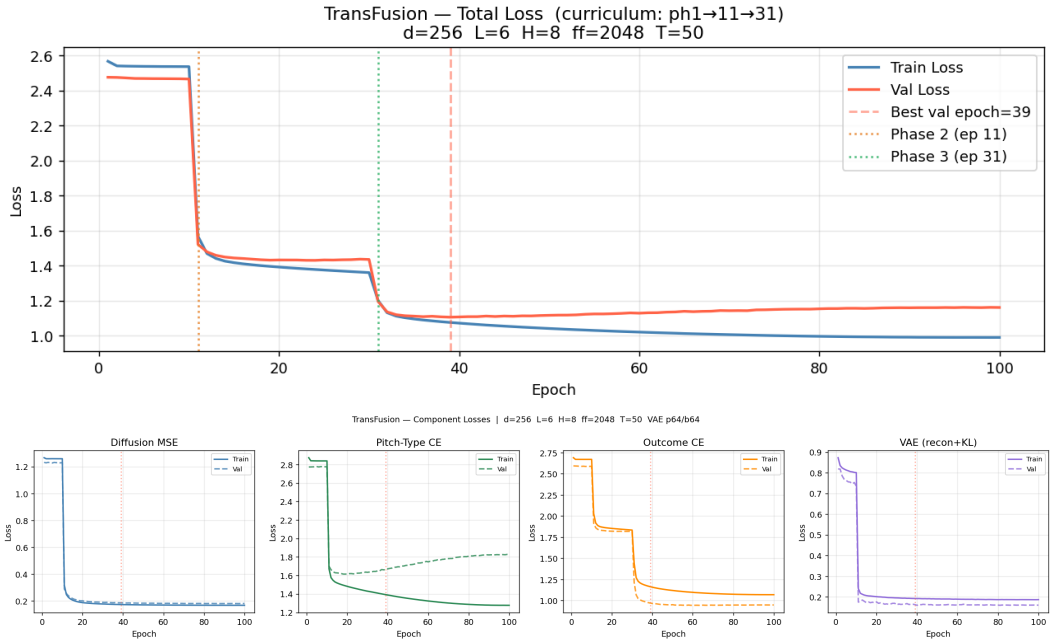

Figure 2: Training loss curves for TransFusion.

3.3 Simulation-Based Probability Estimation

3.3.1 Plain Monte Carlo Simulation

Our ultimate objective is not only to generate realistic pitch sequences, but to use those generated sequences to estimate conditional win probabilities from a partially observed game. To do so, we begin with the observed context $\mathcal{C}_k$, encode it with the causal Transformer, and then simulate the remainder of the game autoregressively, one pitch at a time.

More precisely, each simulation proceeds as follows. Starting from the encoded context and the reconstructed game state at the context boundary, we repeatedly (1) sample the next pitch type from the pitch-type head, (2) generate the next pitch's continuous features with the conditional diffusion decoder, (3) sample the next pitch outcome from the outcome head, and (4) update the game state deterministically according to the baseball rules engine.

This rollout continues until the game terminates under standard baseball rules, including extra innings and walk-off termination.

If we generate K simulated continuations $\tau^{(1)}, \ldots, \tau^{(K)}$ then we estimate the home-team win probability by the empirical average

$$\widehat{p}_{\text{win}}(\mathcal{C}_k) = \frac{1}{K} \sum_{i=1}^{K} \mathbf{1}\left[\text{home_win}\left(\tau^{(i)}\right)\right]. \tag{5}$$

In addition to win probability, we record the mean and standard deviation of simulated home and away runs. This allows us to evaluate not only winner prediction, but also the model's ability to reconstruct the final box score.

3.3.2 Metropolis–Hastings MCMC

Plain Monte Carlo uses the model distribution $P_\theta(\tau \mid \mathcal{C}_k)$ directly. However, even if the simulated trajectories are locally realistic, they may still underrepresent or overrepresent certain run-scoring patterns over the remainder of the game. To account for this, we also study a Metropolis–Hastings correction that tilts the simulation distribution toward trajectories whose base/out states are more consistent with historical run expectancy.

Target distribution. We define the target distribution by

$$\pi(\tau \mid \mathcal{C}_k) \propto P_\theta(\tau \mid \mathcal{C}_k) \exp\left(\lambda \sum_j \phi(s_j)\right), \tag{6}$$

where s_j denotes the base/out state at step j, $\lambda \geq 0$ controls the strength of the correction, and

$$\phi(s_j) = \log \mathrm{RE24}(s_j) - \log \max_s \mathrm{RE24}(s) \leq 0 \tag{7}$$

is a log-normalized run-expectancy score over the 24 base/out states. When $\lambda = 0$, the target reduces to the uncorrected model distribution.

Proposal mechanism. Our proposal distribution is based on suffix resimulation. Given a current simulated trajectory τ, we choose a random half-inning boundary, keep the trajectory fixed up to that boundary, reconstruct the corresponding game state and context representation, and then resimulate the suffix from that point onward. This proposal is appealing for two reasons: it preserves valid baseball state transitions, and it allows us to modify a substantial portion of the future trajectory while still conditioning on a realistic prefix.

Acceptance and estimation. Let τ' denote the proposed trajectory obtained by resimulating from a chosen half-inning boundary b. The resulting acceptance probability is

$$\alpha(\tau, \tau') = \min\left(1,\ \frac{m(\tau)}{m(\tau')} \exp\left(\lambda \sum_{j \geq b} \left[\phi(s'_j) - \phi(s_j)\right]\right)\right), \tag{8}$$

where $m(\tau)$ denotes the number of admissible half-inning boundaries in trajectory τ. The ratio $m(\tau)/m(\tau')$ accounts for the asymmetry of the suffix-resimulation proposal, while the exponential term compares the RE24-weighted quality of the proposed and current suffixes. In particular, the model likelihood cancels from the acceptance ratio, so the correction can be computed using only the proposal-boundary counts and RE24 table lookups.

After a burn-in period of $B = 100$ steps, we estimate win probability from the next $T = 500$ states of the Markov chain:

$$\widehat{p}_{\mathrm{win}}(\mathcal{C}_k) = \frac{1}{T} \sum_{t=1}^{T} \mathbf{1}\left[\mathrm{home_win}\left(\tau^{(t)}\right)\right]. \tag{9}$$

When $\lambda = 0$, every proposal is accepted, and the procedure recovers the corresponding uncorrected Monte Carlo estimator.

3.4 Baseline MLP

As a baseline, we train a direct win-probability classifier that predicts the game winner from a fixed summary of the observed context. We train a separate three-layer MLP for each context inning and compare its performance against TransFusion at the corresponding context level. Unlike the generative model, the MLP predicts only the final winner and does not generate score trajectories or run estimates. Each MLP is trained with dropout $p = 0.25$, learning rate 0.001, and the Adam optimizer.

4 Results

We evaluate TransFusion and the baseline MLP across all context innings (0 through 8.5) on three metrics: prediction accuracy, Brier score, and log loss. Accuracy measures the fraction of games

where the higher-probability team wins; Brier score captures calibration by penalizing the squared difference between predicted probabilities and binary outcomes (lower is better); and log loss more heavily penalizes confident incorrect predictions (lower is better). For each context level, we report the mean and ±1 standard error across test-set games.

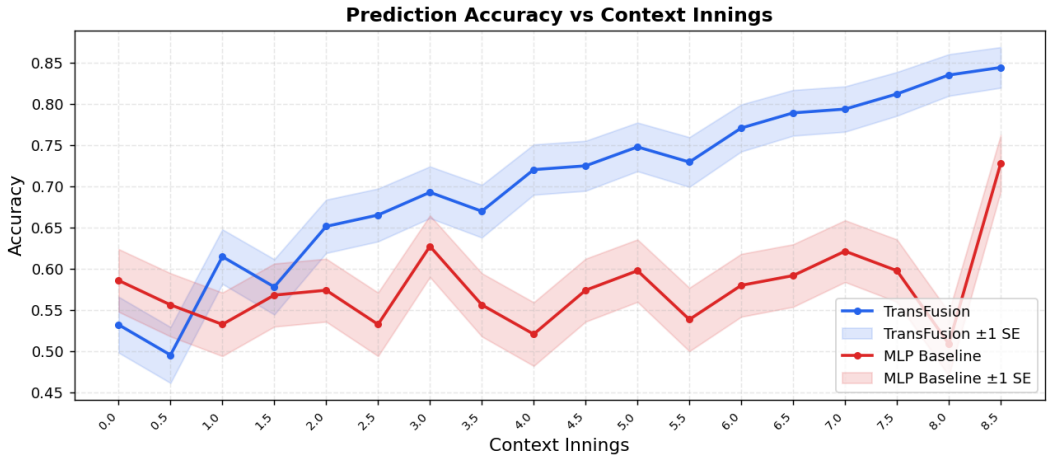

Figure 3: Prediction Accuracy

Prediction Accuracy (Figure 3). At 0 context innings, both models hover near 50%, confirming that pre-game features alone (team records, weather, matchup metadata) carry limited discriminative power — consistent with MLB's well-documented competitive parity. TransFusion begins to separate from the MLP baseline around 1.5 context innings and maintains a consistent advantage thereafter, reaching approximately 82% accuracy at 8.5 innings compared to roughly 75% for the MLP. The gap is most pronounced in the mid-game range (3.5–6.5 innings), suggesting that TransFusion's pitch-level trajectory simulation captures compounding in-game dynamics — such as pitcher fatigue and baserunner effects — that the MLP's fixed-dimensional state summary discards.

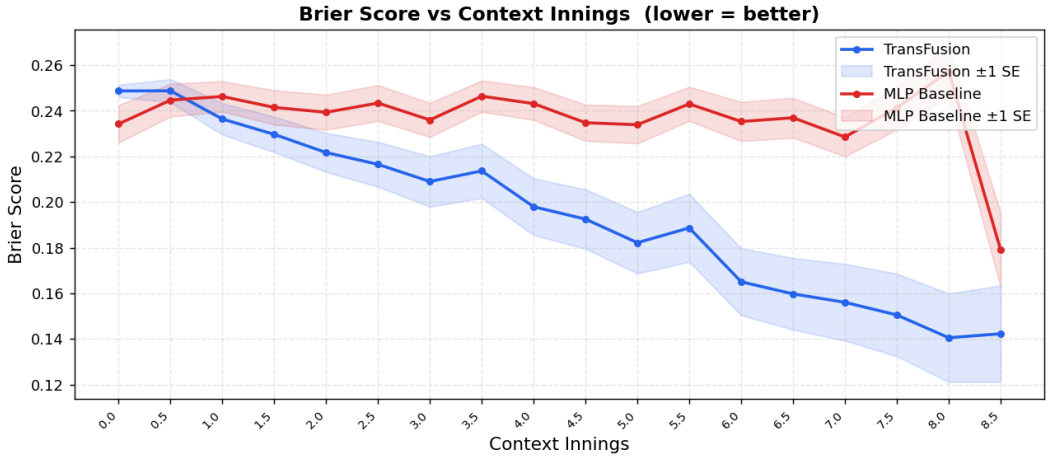

Figure 4: Brier Score

Brier Score (Figure 4). Brier score trends mirror accuracy, with both models starting near 0.25 (the expected value for a uniform 50/50 predictor) and declining as context increases. TransFusion achieves consistently lower Brier scores at every context level, indicating better-calibrated probability estimates. By 8.0 innings, TransFusion reaches a Brier score below 0.14, while the MLP baseline remains above 0.15. The tighter standard error bands for TransFusion also suggest that its estimates are more stable across games.

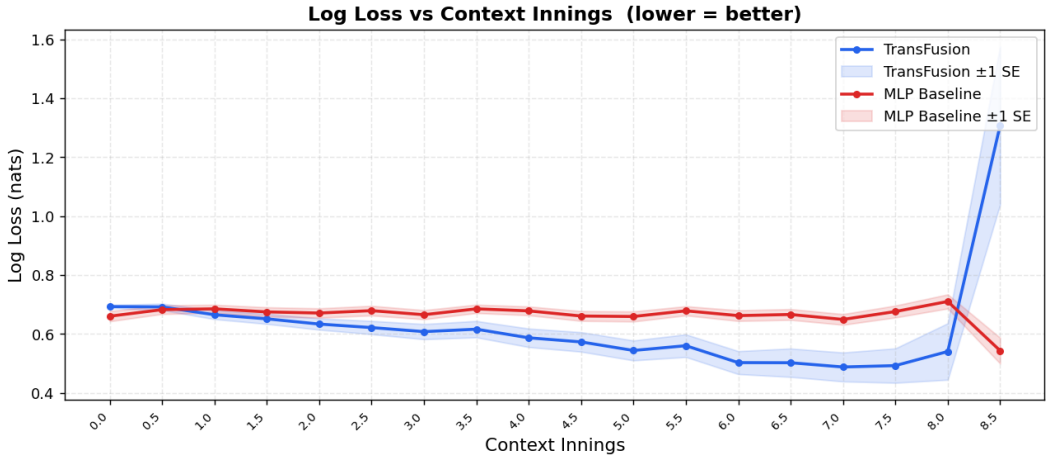

Figure 5: Log Loss

Log Loss (Figure 5). Log loss follows a similar downward trajectory for both models through approximately 7.5 context innings. However, TransFusion exhibits a notable spike at 8.5 innings, rising above its 7.5-inning value even as accuracy continues to improve. This divergence occurs because log loss penalizes confident wrong predictions exponentially: when TransFusion assigns near-certain win probabilities (e.g., 0.95+) and the trailing team mounts a comeback, the resulting penalty is severe. We attribute this to the late-game overconfidence phenomenon described by [BYW25], where estimators systematically underestimate residual variance when most of the game has been observed. The MLP baseline, which produces less extreme probabilities overall, is less susceptible to this effect.

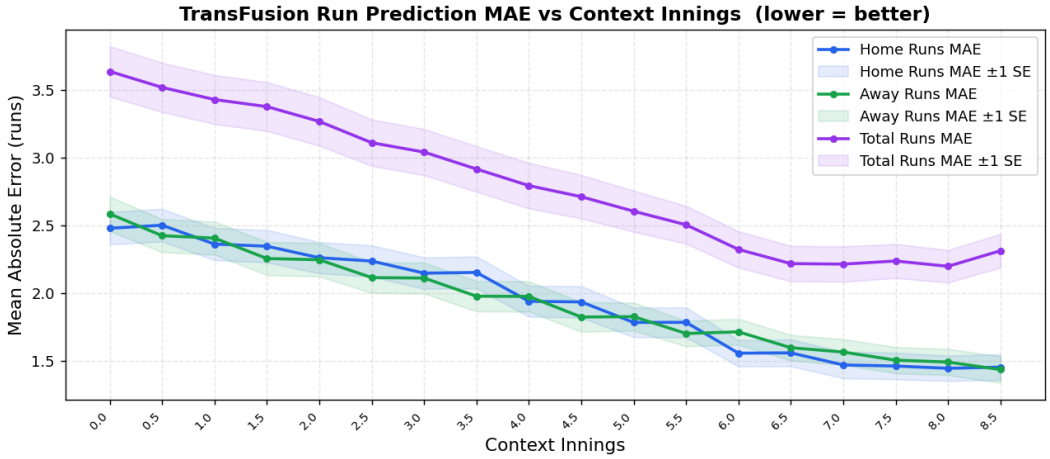

Figure 6: Run Prediction

Run Prediction (Figure 6). TransFusion's ability to predict final run totals — a capability the MLP baseline lacks entirely — is shown in Figure 6. Mean absolute error decreases steadily for home runs, away runs, and total runs as context increases, dropping from roughly 3.5 runs at 0 innings to approximately 1.5 runs by 8.0 innings. However, at 8.5 context innings, the away team's run total should be fully determined (since the away team has completed batting), yet the model still exhibits nonzero prediction error. This indicates underfitting in the outcome classification head, which fails to perfectly reconstruct known game state and propagates incorrect run counts into downstream simulations. Addressing this gap — through additional training data, improved outcome-head capacity, or explicit constraints that lock in completed half-innings — is a priority for future work.

5 Discussion and Analysis

The results suggest that pitch-level generative simulation provides a meaningful advantage over a direct state-summary classifier for baseball win-probability estimation. As more context innings are observed, both models improve, but TransFusion improves more quickly and more consistently on both accuracy and Brier score. This indicates that the gain is not merely in classification, but in probability estimation as well.

5.1 Why Pitch-Level Simulation Helps

We attribute this improvement to the fact that TransFusion models the remainder of the game as a sequential stochastic process rather than as a single fixed state. A state-summary classifier compresses the observed prefix into a finite-dimensional vector and necessarily discards temporal structure, including pitch ordering, count evolution, and within-inning interactions. By contrast, TransFusion preserves these dependencies by generating future pitches autoregressively. This advantage is most visible in the middle innings, where the observed state is informative but the outcome is still far from determined.

5.2 Late-Game Calibration and State Consistency

The sharp increase in log loss at 8.5 context innings reveals a limitation distinct from poor accuracy. Near the end of the game, simulated trajectories can become too concentrated, causing the empirical win fraction to collapse too quickly toward 0 or 1 even when meaningful uncertainty remains. As a result, a small number of highly confident mistakes produces a large log-loss penalty. This is consistent with prior observations that play-by-play win-probability estimates become high-variance when many trajectories share the same observed prefix.

The run-prediction results reveal a related state-consistency issue. At 8.5 innings, the away team's run total should already be fixed, yet the model still exhibits nonzero away-run error. This suggests that the outcome head does not always fully respect known information at the context boundary, and that rollout should enforce stronger consistency constraints in completed half-innings.

5.3 Limitations and Improvements

The main limitations of the current system are data and representation driven. Although Statcast provides a large pitch-level dataset, many important baseball events remain rare, especially in late-game, high-leverage settings. In addition, the current feature set does not fully encode matchup-specific structure such as batter–pitcher history, platoon splits, park effects, and recent form. These omissions likely reduce both sequence fidelity and win-probability calibration.

Our results also suggest that larger models alone are unlikely to resolve these issues. Wider and deeper TransFusion variants did not materially improve validation loss, indicating that better conditioning information may be more important than additional scale. Future improvements should therefore prioritize richer matchup features, stronger state-consistency constraints during rollout, and better proposal or sampling mechanisms for late-game simulation.

5.4 Broader Implications

More broadly, these results support the idea that probability estimation can benefit from modeling the latent trajectory of a system rather than predicting the endpoint directly. Baseball is a natural setting for this because it is already organized as a sequence of discrete, context-dependent events, but the same principle may apply to other sequential forecasting problems in sports and beyond.

References

[BYW25] Ryan S. Brill, Ronald Yurko, and Abraham J. Wyner. "Exploring the Difficulty of Estimating Win Probability: A Simulation Study". In: *arXiv preprint arXiv:2406.16171 [stat.ME]* (2025). version 5. arXiv: 2406.16171 [stat.ME]. URL: https://arxiv.org/abs/2406.16171.

[Cui+25] Ziqiang Cui et al. "Diffusion-based Contrastive Learning for Sequential Recommendation". In: *arXiv preprint arXiv:2405.09369 [cs.IR]* (2025). version 5. arXiv: 2405.09369 [cs.IR]. URL: https://arxiv.org/abs/2405.09369.

[HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851. URL: https://arxiv.org/abs/2006.11239.

[MLBnd] MLB Advanced Media. *Statcast Search*. Baseball Savant. n.d. URL: https://baseballsavant.mlb.com/statcast_search.

[ND21] Alexander Quinn Nichol and Prafulla Dhariwal. "Improved Denoising Diffusion Probabilistic Models". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. 2021, pp. 8162–8171. URL: https://arxiv.org/abs/2102.09672.

[SRH23] Md Fahim Sikder, Resmi Ramachandranpillai, and Fredrik Heintz. "TransFusion: Generating Long, High Fidelity Time Series using Diffusion Models with Transformers". In: *arXiv preprint arXiv:2307.12667 [cs.LG]* (2023). version 2. arXiv: 2307.12667 [cs.LG]. URL: https://arxiv.org/abs/2307.12667.

[SME21] Jiaming Song, Chenlin Meng, and Stefano Ermon. "Denoising Diffusion Implicit Models". In: *International Conference on Learning Representations (ICLR)*. 2021. URL: https://arxiv.org/abs/2010.02502.

[Ste+25] Tristan S. W. Stevens et al. "Sequential Posterior Sampling with Diffusion Models". In: *ICASSP 2025 – 2025 IEEE International Conference on Acoustics, Speech and Signal Processing*. 2025. DOI: 10.1109/ICASSP49660.2025.10889752. URL: https://doi.org/10.1109/ICASSP49660.2025.10889752.

[Vas+17] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008. URL: https://arxiv.org/abs/1706.03762.

[Zho+24] Chunting Zhou et al. "Transfusion: Predict the Next Token and Diffuse Images with One Multi-Modal Model". In: *arXiv preprint arXiv:2408.11039 [cs.AI]* (2024). arXiv: 2408.11039 [cs.AI]. URL: https://arxiv.org/abs/2408.11039.